

VitaPair

Documento de Arquitetura e Contexto de Projeto

Versao 2.0 | Para uso com Claude Code

Junho 2025

0. Identidade e Visao do Produto

0.1 O Problema Real

Apps de saude hoje sao planilhas glorificadas. Voce registra, ve um numero, fecha o app e nunca mais volta. MyFitnessPal, Cronometer, FatSecret — todos resolvem o problema errado: focam no dado, nao no comportamento.

- Dar trabalho demais para registrar — friccao alta = abandono rapido
- Ser chato e sem graca — interface de planilha, sem personalidade
- Nao prender a atencao — sem motivo para abrir o app amanha
- Ser generico demais — o mesmo plano para todo mundo
- Nao ter elemento social — saude e tratada como algo solitario

0.2 A Solucao — O Duolingo da Saude para Casais

VitaPair nao e um app de dieta. E um jogo de dois jogadores onde o premio e ficar saudavel juntos. A referencia de produto e o Duolingo: simples de usar, impossivel de largar, com elemento social que cria responsabilidade sem pressao toxica.

Posicionamento do Produto

- NAO somos: app de contagem de calorias
- SOMOS: plataforma de saude gamificada com rivalidade entre pares
- Publico: casais, duplas de amigos, qualquer pessoa que queira dinamica social na jornada de saude
- Tom: energia, leveza, rivalidade carinhosa — sem linguagem medica ou de dieta
- Benchmark: Duolingo (retencao) + Strava (social + competicao) + IA generativa (personalizacao)

0.3 As Quatro Camadas do Produto

Quatro camadas que se empilham. Se qualquer uma falhar, o produto nao funciona:

Camada	Funcao	Se falhar...
1. Simplicidade absurda	Registrar em menos de 10 segundos via foto — sem friccao	Usuario abandona no primeiro dia
2. Historia progressiva	Personagem que evolui, temporadas, aposta — motivo para voltar amanha	Usuario nao tem razao para reabrir o app
3. IA contextual	Plano adaptado em tempo real, sugestoes inteligentes e pessoais	App parece generico igual aos concorrentes
4. Rivalidade saudavel	Competicao com o par, notificacoes provocativas, missoes surpresa	Falta o diferencial que nenhum concorrente tem

0.4 O Conceito Central — Temporadas e Apostas

Cada par inicia uma Temporada de 30 dias. Antes de começar, os dois combinam uma aposta (ex: quem perder paga o jantar). Isso cria stakes reais sem pressao toxica — o objetivo e criar uma historia compartilhada, nao punir.

- Temporada de 30 dias com placar em tempo real
- Aposta customizada definida pelo casal antes de começar
- Pontos por consistencia, nao por perfeicao — a meta e nao desistir
- Historico de temporadas: quantas cada um ganhou ao longo do tempo
- Modo Solo: usuario sem par compete contra a versao de si mesmo da semana passada

0.5 Sistema de Pontuacao

Acao	Pontos	Logica
Registrar refeicao	+10 pts	Qualquer registro conta — sem julgamento
Bater meta calorica do dia	+20 pts	Ficou dentro da janela de +/- 10% da meta
Completar treino	+15 pts	Qualquer treino registrado ou importado
Streak de 3 dias	+30 pts bonus	Consistencia vale mais que perfeicao
Streak de 7 dias	+80 pts bonus	Uma semana seguida = recompensa grande
Missao relampago concluida	+50 pts	Desafios surpresa no meio da semana
Meta semanal de peso atingida	+100 pts	Avanco real em direcao ao objetivo
Reagir ao registro do par	+5 pts	Incentivar o elemento social

0.6 Personagem Evolutivo

Cada usuario tem um personagem visual simples que reflete sua consistencia — nao punitivo, mas como um espelho. Design minimalista, nao infantil:

- Nivel 1-5 Iniciante: personagem em repouso, cores neutras
- Nivel 6-15 Ativo: personagem com mais energia, cores vibrantes
- Nivel 16-30 Atleta: personagem com aura de conquista
- Inatividade 3+ dias: personagem adormecido — pressao sutil para voltar
- Vitoria de temporada: animacao especial no perfil

0.7 Registro Ultra-Rapido — Menos de 10 Segundos

- Foto da refeicao: IA identifica alimentos e estima calorias automaticamente (Anthropic API)
- Busca rapida: digitar o nome e selecionar porcao — autocomplete Open Food Facts
- Favoritos: refeicoes frequentes em 1 toque
- Importacao automatica: WeWard e Google Fit alimentam atividade sem acao manual
- Voz (v2 futuro): descrever a refeicao em audio, IA calcula

0.8 Notificacoes Provocativas

Tom de rivalidade carinhosa — nunca culpa, nunca pressao negativa:

- Ana acabou de registrar o almoco. Voce esta perdendo por 40 pontos. Vai ficar assim?
- MISSAO RELAMPAGO: quem beber 2L de agua primeiro ganha 50 pontos. Comeca agora!
- Sao 20h e voce nao registrou nada hoje. Seu streak de 5 dias esta em risco.
- Joao treinou hoje. Voce nao. Placar: 340 x 290. Ele esta chegando.
- Parabens! Voce bateu sua meta pela 3a vez seguida. Seu par esta com inveja.

0.9 Identidade Visual — Diretrizes para o Claude Code

Elemento	Diretriz	Evitar
Paleta	Vibrante — laranja, roxo, verde neon como acentos sobre fundo escuro ou claro	Verde hospitalar, azul clinico, branco esteril
Tipografia	Sans-serif moderna arredondada — proximidade, nao seriedade	Fontes serifadas ou medicas
Icones	Preenchidos, arredondados, com personalidade	Icones lineares genericos
Graficos	Animados, com celebracao ao atingir metas	Graficos estaticos de planilha
Linguagem	Informal e brasileiro: mandou bem, bora la, top demais	Terminologia medica ou ingles desnecessario
Layout	Generoso, uma acao principal por tela, sem poluicao visual	Dashboards com excesso de informacao
Estados vazios	Provocativos e divertidos quando nao ha dados	Telas em branco sem personalidade

1. Visao Geral Tecnica

VitaPair e um aplicativo web de saude e fitness gamificado para casais e duplas. O produto combina quatro pilares: registro ultra-rapido com IA, historia progressiva com personagem evolutivo, plano adaptativo em tempo real, e competicao entre pares. Stack: Java 17 + Spring Boot 3 no backend, React + TypeScript + Tailwind no frontend, PostgreSQL + Redis no banco, Oracle Free Tier na infra.

1.1 Publico-Alvo

- Primario: casais com objetivos individuais diferentes (ela perde peso, ele ganha massa)
- Secundario: duplas de amigos, irmaos, colegas que querem competir de forma saudavel
- Terciario: usuarios solo que competem contra a versao de si mesmos da semana anterior
- Perfil BR: familiarizados com WeWard, 20-40 anos, ja tentaram apps de saude e desistiram

1.2 Diferenciais Competitivos

Funcionalidade	VitaPair	MyFitnessPal	Strava
Registro por foto com IA	Sim	Nao	Nao
Competicao entre pares com aposta	Sim (temporadas 30 dias)	Nao	Parcial
Personagem evolutivo	Sim	Nao	Nao
Plano adaptado por IA toda semana	Sim	Nao	Nao
Notificacoes de rivalidade carinhosa	Sim	Nao	Parcial
Missoes relampago surpresa	Sim	Nao	Nao
Modo solo vs si mesmo	Sim	Parcial	Sim
Integracao WeWard e wearables	Sim	Parcial	Sim
Freemium para comecar	Sim	Sim	Sim

1.3 Visao SaaS

Roadmap de Evolucao

- MVP: O proprio casal como primeiros usuarios — custo zero com Oracle Free Tier

- Beta: Abrir para outros casais e duplas — multi-tenancy preparado desde o início
- SaaS Freemium: 1 par + funções básicas grátis | Premium: IA avançada, histórico ilimitado
- Monetização adicional: planos premium gerados por IA, parcerias com nutricionistas
- Escala: migrar Oracle para Hetzner ou DigitalOcean conforme crescimento
- Stripe para pagamentos e assinaturas recorrentes

2. Stack Tecnica

2.1 Decisoões Tecnológicas

Camada	Tecnologia	Justificativa
Frontend	React + TypeScript + Tailwind CSS	Ecosistema robusto, tipagem segura, styling produtivo
Backend	Java 21 + Spring Boot 3	Mercado BR, performance, ecossistema maduro, Spring Security
ORM	Spring Data JPA + Hibernate	Padrao de mercado com Java/Spring
Banco principal	PostgreSQL 15+	Relacional, confiavel, suporte a JSON, extensions uteis
Cache / Sessoes	Redis	Sessoes JWT, cache de TDEE, streaks, rate limiting
Auth	Spring Security + JWT + OAuth2	Login proprio + Google/Apple login
API de Alimentos	Open Food Facts	Gratuita, ampla cobertura de produtos BR, sem limite de requests
Infra	Oracle Free Tier VPS + DuckDNS	Custo zero para MVP: 2 OCPUs, 12GB RAM, 200GB disco
Reverse proxy	Nginx	SSL, balanceamento, servir frontend estatico
Containers	Docker + Docker Compose	Ambiente reproduzivel, deploy simples
CI/CD	GitHub Actions	Automatizar build, testes e deploy no Oracle VPS

2.2 Por que nao Vercel para o Backend

Vercel e otimo para frontends e APIs serverless simples, mas tem restricoes criticas para um SaaS com backend Java:

- Nao suporta Spring Boot nativamente — requer containers customizados
- Cobranca por invocacao de funcao: escala de custo imprevisivelmente
- Sem controle total do servidor — impossivel rodar Redis, workers, jobs agendados
- Multi-tenancy com banco proprio e muito mais complexo de gerenciar
- Decisao: Oracle Free Tier durante MVP, migracao planejada para Hetzner/DigitalOcean ao virar SaaS

2.3 Plano de Migracao de Infra

Fase	Infra	Custo Estimado	Gatilho
MVP / Desenvolvimento	Oracle Free Tier + DuckDNS	R\$ 0	Ate ~100 usuarios ativos
SaaS inicial	Oracle + dominio proprio (.app)	~R\$ 50/ano	Primeiro pagante
Crescimento	Hetzner CX21 (VPS 4GB) + Cloudflare Free	~R\$ 50/mes	Acima de 500 usuarios
Escala	DigitalOcean Managed DB + auto-scaling	~R\$ 200+/mes	Acima de 5.000 usuarios

3. Arquitetura do Sistema

3.1 Visao de Alto Nivel

O sistema segue arquitetura monolitica modular no backend — adequada para o estagio atual — com separacao clara de responsabilidades por pacote. A decisao consciente e nao sobre-engenheirar com microservicos antes de ter produto validado; a estrutura de pacotes, entretanto, ja esta organizada para facilitar a extracao futura de servicos se necessario.

Fluxo de Requisicao (simplificado)

- Usuario acessa via browser → Nginx serve o React (build estatico)
- React faz chamadas REST para /api/* → Nginx faz proxy para Spring Boot (porta 8080)
- Spring Boot autentica JWT → processa logica de negocio → consulta PostgreSQL ou Redis
- Para busca de alimentos: Spring Boot chama Open Food Facts API externamente
- Para dados de wearables: integracao OAuth2 com Google Fit / Strava / etc.

3.2 Estrutura de Pacotes — Backend (Java)

Organizacao por feature/dominio, nao por camada tecnica. Facilita manutencao e possivel extracao para microservicos:

vitapair-backend/

```
src/main/java/com/vitapair/  
  config/          # SecurityConfig, RedisConfig, JacksonConfig, CorsConfig  
  common/          # BaseEntity, ApiResponse, GlobalExceptionHandler, PageResponse  
  auth/            # AuthController, AuthService, JwtService, OAuth2Handler  
    dto/           # LoginRequest, RegisterRequest, TokenResponse  
  user/            # UserController, UserService, UserRepository  
    domain/        # User.java (entidade JPA)  
    dto/           # UserProfileDto, UpdateProfileRequest  
  pair/            # PairController, PairService, PairRepository  
    domain/        # Pair.java - relacionamento entre dois usuarios  
    dto/           # PairInviteRequest, PairStatusDto  
  nutrition/       # NutritionController, NutritionService, FoodLogRepository  
    domain/        # FoodLog.java, Meal.java, FoodItem.java  
    dto/           # FoodLogRequest, DailyNutritionSummary  
    integration/   # OpenFoodFactsClient.java (Feign ou RestTemplate)  
  workout/         # WorkoutController, WorkoutService, WorkoutPlanRepository  
    domain/        # WorkoutPlan.java, Exercise.java, WorkoutSession.java  
    dto/           # WorkoutPlanDto, LogSessionRequest  
  tdee/            # TdeeService - calculo de TDEE, BMR, macro targets  
  activity/        # ActivityController, ActivityService  
    domain/        # ActivityLog.java - passos, km, calorias de wearables  
    integration/   # WearableIntegration (WeWard, GoogleFit, Strava)  
  gamification/    # GamificationService, BadgeRepository, StreakService  
    domain/        # Badge.java, UserStreak.java, CompetitionScore.java  
  ai/              # AiPlanService - chamadas para LLM (geracao de planos)  
  tenant/          # TenantContext, TenantFilter - multi-tenancy (SaaS ready)  
src/main/resources/
```

```
application.yml
application-dev.yml
application-prod.yml
db/migration/      # Flyway migrations (V1__init.sql, V2__add_pairs.sql, ...)
```

3.3 Estrutura do Frontend (React)

```
vitapair-frontend/
src/
  api/          # axios instance, interceptors, endpoints por feature
  components/   # componentes reutilizaveis (Button, Card, Chart, Modal...)
  features/
    auth/       # Login, Register, OAuth2Callback
    dashboard/  # DashboardPage, CalorieRing, MacroSummary
    pair/       # PairFeed, PartnerCard, CompetitionScore
    nutrition/  # FoodSearch, MealLog, DailyNutritionView
    workout/    # WorkoutPlan, ExerciseList, LogSession
    activity/   # StepsInput, WearableConnect, ActivityHistory
    profile/    # ProfileSetup, GoalConfig, TdeeDisplay
    gamification/ # Badges, Streaks, Leaderboard
  hooks/        # useAuth, usePair, useNutrition, useWorkout
  store/        # Zustand ou Context API para estado global
  types/        # TypeScript interfaces globais
  utils/        # formatters, calculators, validators
  router/       # React Router - rotas publicas e protegidas
```

4. Modelagem do Banco de Dados

4.1 Principio de Multi-Tenancy

Mesmo sendo um MVP para dois usuarios, o banco ja deve ser modelado pensando em multi-tenancy para nao precisar de uma reengenharia cara ao virar SaaS. A abordagem escolhida e shared database, shared schema com coluna tenant_id em todas as tabelas de dados. Cada par de usuarios forma um "tenant".

Estrategia de Multi-Tenancy

- Abordagem: Shared database, shared schema (mais simples de operar no inicio)
- Isolamento: coluna tenant_id em todas as entidades de negocio
- Garantia: TenantFilter intercepta toda requisicao e injeta tenant_id automaticamente via ThreadLocal
- Spring Data: @Query sempre filtra por tenant_id — nunca vaza dados entre tenants
- Futuro: migracao para schemas separados (uma linha de mudanca) se compliance exigir

4.2 Entidades Principais

users

Coluna	Tipo	Descricao
id	UUID PK	Identificador unico
tenant_id	UUID NOT NULL	Referencia ao pair/tenant
email	VARCHAR(255) UNIQUE	Login do usuario
password_hash	VARCHAR(255)	BCrypt — nulo se OAuth2
name	VARCHAR(100)	Nome de exibicao
birth_date	DATE	Para calculo de idade no TDEE
sex	ENUM(MALE, FEMALE, OTHER)	Para calculo de BMR (Harris-Benedict)
height_cm	DECIMAL(5,2)	Altura em centimetros
weight_kg	DECIMAL(5,2)	Peso atual em kg
goal	ENUM(LOSE_WEIGHT, GAIN_MUSCLE, MAINTAIN, IMPROVE_FITNESS)	Objetivo principal

activity_level	ENUM(SEDENTARY, LIGHT, MODERATE, ACTIVE, VERY_ACTIVE)	Para multiplicador TDEE
daily_calorie_target	INT	Calculado automaticamente pelo TdeeService
protein_target_g	INT	Meta de proteina em gramas
carb_target_g	INT	Meta de carboidrato em gramas
fat_target_g	INT	Meta de gordura em gramas
avatar_url	VARCHAR(500)	Foto de perfil
created_at	TIMESTAMP	Auto-preenchido
updated_at	TIMESTAMP	Auto-atualizado

pairs (o "tenant" do sistema)

Coluna	Tipo	Descricao
id	UUID PK	O tenant_id referenciado em todas as outras tabelas
user1_id	UUID FK -> users	Primeiro membro do par
user2_id	UUID FK -> users	Segundo membro (pode ser nulo ate aceitar convite)
pair_name	VARCHAR(100)	Nome do par (ex: "Alisson & Ana")
invite_code	VARCHAR(20) UNIQUE	Codigo unico para convite
status	ENUM(PENDING, ACTIVE, PAUSED)	Estado da parceria
created_at	TIMESTAMP	

food_logs (registro diario de refeicoes)

Coluna	Tipo	Descricao
id	UUID PK	
tenant_id	UUID NOT NULL	Isolamento multi-tenant
user_id	UUID FK -> users	Quem comeu
food_name	VARCHAR(255)	Nome do alimento
barcode	VARCHAR(50)	Codigo de barras (Open Food Facts)
quantity_g	DECIMAL(7,2)	Quantidade em gramas

calories_kcal	DECIMAL(7,2)	Calorias totais da porcao
protein_g	DECIMAL(6,2)	
carb_g	DECIMAL(6,2)	
fat_g	DECIMAL(6,2)	
meal_type	ENUM(BREAKFAST, LUNCH, DINNER, SNACK)	
logged_at	TIMESTAMP	Data/hora da refeicao
source	ENUM(OPEN_FOOD_FACTS, MANUAL)	Origem do dado

activity_logs (passos, exercicios, calorias gastas)

Coluna	Tipo	Descricao
id	UUID PK	
tenant_id	UUID NOT NULL	
user_id	UUID FK -> users	
activity_type	ENUM(STEPS, RUN, WALK, CYCLE, WORKOUT, OTHER)	
steps	INT	Passos totais (se aplicavel)
distance_km	DECIMAL(6,3)	Distancia percorrida
calories_burned	DECIMAL(7,2)	Calorias gastas calculadas
duration_minutes	INT	Duracao em minutos
source	ENUM(WEWARD, GOOGLE_FIT, APPLE_HEALTH, STRAVA, GARMIN, MANUAL)	
external_id	VARCHAR(255)	ID na plataforma de origem (evita duplicatas)
logged_at	TIMESTAMP	

workout_plans e workout_sessions

Tabela	Principais colunas	Descricao
workout_plans	id, tenant_id, user_id, name, goal, generated_by_ai, is_active	Plano de treino gerado (manual ou por IA)

workout_exercises	id, plan_id, name, sets, reps, rest_seconds, order_index	Exercicios dentro de um plano
workout_sessions	id, tenant_id, user_id, plan_id, calories_burned, notes, completed_at	Registro de treino realizado

gamification (badges, streaks, pontuacao)

Tabela	Principais colunas	Descricao
badges	id, code, name, description, icon_url, category	Catalogo de conquistas disponiveis
user_badges	id, tenant_id, user_id, badge_id, earned_at	Badges conquistados por usuario
user_streaks	id, tenant_id, user_id, type, current_count, longest_count, last_activity_date	Sequencias diarias (login, treino, log alimentar)
competition_scores	id, tenant_id, week_start, user1_score, user2_score, winner_id	Placar semanal entre o par

5. Funcionalidades Detalhadas

5.1 Onboarding e Perfil

- Cadastro com email/senha ou OAuth2 (Google, Apple)
- Formulário de perfil: nome, data nascimento, sexo, altura, peso atual, objetivo, nível de atividade
- Cálculo automático de TDEE ao salvar perfil (Harris-Benedict + multiplicador de atividade)
- Definição de macros: proteína (2g/kg para ganho de massa, 1.6g/kg para perda), carbô e gordura calculados do restante
- Convite para parceiro: gera código de 8 dígitos único, parceiro aceita e o par é criado

5.2 Dashboard Diário

- Anel de calorias: consumidas vs meta (gráfico circular animado)
- Macros do dia: barras de progresso para proteína, carbô e gordura
- Calorias gastas: soma de atividades do dia (passos + treino)
- Balanço final: consumido - gasto vs meta (positivo = superavit, negativo = déficit)
- Resumo do parceiro: mini-card mostrando o progresso do par no mesmo dia
- Alertas inteligentes: "Você está 300kcal abaixo da sua meta, considere um lanche"

5.3 Registro de Refeições — Open Food Facts

- Busca por nome do produto: autocomplete chamando Open Food Facts API
- Busca por código de barras: input manual ou câmera do celular (via API do browser)
- Seleção de porção: quantidade em gramas com cálculo automático de macros
- Tipo de refeição: café da manhã, almoço, jantar ou lanche
- Entrada manual: para alimentos caseiros sem registro na base
- Cache de produtos mais usados no Redis (evitar chamadas repetidas a API externa)

5.4 Atividade Física e Integração WeWard

- Input manual de passos com cálculo de calorias (fórmula: passos x 0.04 kcal para pessoa média)
- Input manual de calorias gastas (para quem sabe o valor do WeWard diretamente)
- Webhook/polling para WeWard quando API estiver disponível
- Registro de treino: selecionar plano, marcar como concluído, registrar calorias gastas
- Histórico de atividades com gráficos de progresso semanal/mensal

5.5 Feed Compartilhado do Par

- Timeline cronologica mostrando acoes dos dois usuarios
- Cada item do feed: refeicao logada, treino concluido, meta diaria atingida, novo badge
- Reacoes: botoes de "fogo" (motivacao), "olho" (saudade), "forca" (incentivo)
- Visibilidade controlada: usuario pode marcar refeicoes como privadas
- Notificacoes: "Seu par acabou de treinar!" (via browser push notifications)

5.6 Competicao e Gamificacao

- Placar semanal: pontos por consistencia, nao por quem e mais fit
- Sistema de pontos: log alimentar completo (+10pts), treino concluido (+15pts), meta diaria atingida (+20pts), streak de 7 dias (+50pts)
- Streaks: dias consecutivos com log alimentar, treinos, metas atingidas
- Badges por categoria: nutricao, treino, consistencia, social, marcos de peso
- Desafios semanais do par: "Beba 2L de agua por 5 dias", "Treine 3x essa semana juntos"
- Historico de campeonatos: quem ganhou cada semana

5.7 Planos Gerados por IA

- Plano alimentar semanal: 7 dias de sugestoes de refeicoes respeitando macros e preferencias
- Plano de treino: baseado no objetivo (perda de gordura = cardio + deficit; ganho = forca + superavit)
- Ajuste periodico: check-in semanal com peso atual, plano e recalibrado automaticamente
- Restricoes alimentares: vegetariano, sem gluten, alergias — configuravel no perfil
- Implementacao: chamada para API Anthropic Claude (claude-sonnet-4-6) com contexto do usuario

6. APIs e Integracao

6.1 API REST — Endpoints Principais

Autenticacao — /api/v1/auth

Metodo	Endpoint	Descricao
POST	/register	Cadastro com email + senha
POST	/login	Login, retorna access_token + refresh_token
POST	/refresh	Renovar token usando refresh_token
POST	/oauth2/google	Login com Google
POST	/logout	Invalidar tokens no Redis

Usuarios — /api/v1/users

Metodo	Endpoint	Descricao
GET	/me	Perfil do usuario autenticado
PUT	/me	Atualizar perfil (peso, objetivo, etc.)
GET	/me/tdee	TDEE e macros calculados
GET	/me/stats	Resumo: peso atual, perda/ganho total, dias ativos

Par — /api/v1/pair

Metodo	Endpoint	Descricao
POST	/invite	Gerar codigo de convite
POST	/join/:code	Aceitar convite e criar par
GET	/	Info do par atual (ambos usuarios)
GET	/feed	Timeline compartilhada (paginada)
GET	/competition	Placar semanal atual

Nutricao — /api/v1/nutrition

Metodo	Endpoint	Descricao
GET	/foods/search?q=:query	Buscar alimento na Open Food Facts
GET	/foods/barcode/:code	Buscar por codigo de barras
GET	/logs?date=:date	Logs do dia

POST	/logs	Registrar refeicao
DELETE	/logs/:id	Remover registro
GET	/summary?date=:date	Resumo diario: calorias, macros, balanço
GET	/history?days=30	Historico de N dias para graficos

Atividade — /api/v1/activity

Metodo	Endpoint	Descricao
POST	/logs	Registrar atividade (passos, treino, cardio)
GET	/logs?date=:date	Atividades do dia
GET	/summary?date=:date	Total de calorias gastas no dia
POST	/wearable/sync	Sincronizar dados de wearable (Google Fit, etc.)

6.2 Open Food Facts — Integracao

Endpoint base: <https://world.openfoodfacts.org/api/v2/product/:barcode.json>

Busca por nome: https://world.openfoodfacts.org/cgi/search.pl?search_terms=:query&json=true

- Nenhuma autenticao necessaria — totalmente gratuita
- Campos extraídos: product_name, nutriments.energy-kcal_100g, proteins_100g, carbohydrates_100g, fat_100g
- Cache no Redis com TTL de 24h para produtos ja consultados (chave: off:barcode:<code>)
- Fallback para banco local de alimentos comuns BR caso a API esteja indisponivel
- User-Agent obrigatorio nos headers: "VitaPair/1.0 (contact@vitapair.app)"

6.3 Calculo de TDEE — Algoritmo

Formula Harris-Benedict (revisada Mifflin-St Jeor)

- Homem: $BMR = (10 \times \text{peso_kg}) + (6.25 \times \text{altura_cm}) - (5 \times \text{idade}) + 5$
- Mulher: $BMR = (10 \times \text{peso_kg}) + (6.25 \times \text{altura_cm}) - (5 \times \text{idade}) - 161$
- $TDEE = BMR \times \text{multiplicador de atividade}$
- Sedentario: x1.2 | Levemente ativo: x1.375 | Moderadamente ativo: x1.55 | Muito ativo: x1.725 | Extremamente ativo: x1.9
- Meta calorica para perda: $TDEE - 500\text{kcal}$ (deficit moderado, ~0.5kg/semana)
- Meta calorica para ganho: $TDEE + 300\text{kcal}$ (superavit controlado para minimizar gordura)
- Macros para perda: proteina 2g/kg, gordura 0.8g/kg, carbo = restante

- Macros para ganho: proteína 2.2g/kg, gordura 1g/kg, carbo = restante

7. Seguranca

7.1 Autenticacao e Autorizacao

- JWT com tempo de expiracao curto: access_token (15 minutos), refresh_token (30 dias)
- Refresh tokens armazenados no Redis com possibilidade de revogacao imediata (logout real)
- Spring Security configura rotas publicas (/auth/**) e protegidas (todo o resto)
- @PreAuthorize nas controllers para garantir que usuario so acesse seus proprios dados
- OAuth2 com Google: validar id_token no backend, nao confiar so no frontend
- Senhas: BCrypt com strength 12 — lento por design para dificultar brute force

7.2 Multi-Tenancy e Isolamento de Dados

- TenantFilter (OncePerRequestFilter): extrai tenant_id do JWT e injeta em ThreadLocal
- TenantContext: classe utilitaria para acessar tenant_id atual em qualquer camada
- Repositorios Spring Data: todos os metodos filtram por tenant_id automaticamente
- Teste de seguranca: garantir que /api/v1/nutrition/logs nunca retorne dados de outro tenant

7.3 Outras Praticas

- HTTPS obrigatorio: certificado Let's Encrypt configurado no Nginx (renovacao automatica via certbot)
- CORS configurado explicitamente: so aceitar requisicoes do dominio do frontend
- Rate limiting no Redis: max 100 requests/minuto por usuario autenticado
- Validacao de input: Bean Validation (@Valid, @NotNull, @Size) em todos os DTOs
- SQL Injection: impossivel com Spring Data JPA / queries parametrizadas
- Logs de seguranca: registrar tentativas de acesso nao autorizado com IP
- Variaveis de ambiente: NUNCA hardcodar senhas, secrets ou API keys no codigo
- Arquivo .env no servidor, injetado como variaveis de ambiente no Docker

8. Infraestrutura e Deploy

8.1 Oracle Free Tier — Configuracao

- Instancia: VM.Standard.A1.Flex (ARM) — 2 OCPUs, 12GB RAM, 200GB Block Storage
- SO: Ubuntu 22.04 LTS
- DNS: DuckDNS para dominio gratuito (vitapair.duckdns.org) durante desenvolvimento
- Dominio proprio: adquirir .app ou .com.br quando for lancar para publico
- Firewall Oracle: abrir apenas portas 22 (SSH), 80 (HTTP) e 443 (HTTPS)

8.2 Docker Compose — Servicos

```
# docker-compose.yml
services:
  postgres:
    image: postgres:15-alpine
    volumes: [postgres_data:/var/lib/postgresql/data]
    environment: [POSTGRES_DB, POSTGRES_USER, POSTGRES_PASSWORD]

  redis:
    image: redis:7-alpine
    command: redis-server --requirepass ${REDIS_PASSWORD}

  backend:
    build: ./vitapair-backend
    depends_on: [postgres, redis]
    environment: [DATABASE_URL, REDIS_URL, JWT_SECRET, ...]
    ports: ["8080:8080"]

  frontend:
    build: ./vitapair-frontend
    ports: ["3000:80"] # Nginx interno serve o build do React

  nginx:
    image: nginx:alpine
    ports: ["80:80", "443:443"]
    volumes: [./nginx.conf:/etc/nginx/nginx.conf, certbot_certs:/certs]
    depends_on: [backend, frontend]
```

8.3 CI/CD com GitHub Actions

- Push para branch main dispara o pipeline
- Jobs: checkout -> build Maven -> testes -> build Docker image -> push para GHCR -> SSH no Oracle e docker-compose pull + up
- Secrets no GitHub: ORACLE_SSH_KEY, ORACLE_HOST, GHCR_TOKEN
- Zero-downtime: docker-compose up --no-deps --build backend (sem derrubar banco e Redis)
- Rollback: manter as 2 ultimas imagens tagueadas para rollback rapido

8.4 Configuracao Nginx

```
# nginx.conf (simplificado)
server {
    listen 443 ssl;
    server_name vitapair.duckdns.org;

    # Frontend (React)
    location / {
        proxy_pass http://frontend:80;
    }

    # Backend (Spring Boot)
    location /api/ {
        proxy_pass http://backend:8080;
        proxy_set_header Authorization $http_authorization;
    }

    ssl_certificate      /certs/live/vitapair.duckdns.org/fullchain.pem;
    ssl_certificate_key  /certs/live/vitapair.duckdns.org/privkey.pem;
}
```

9. Instrucoes para o Claude Code

9.1 O que Este Projeto E

VitaPair e um aplicativo web de saude e fitness para casais. Backend em Java 21 + Spring Boot 3, frontend em React + TypeScript + Tailwind CSS, banco PostgreSQL + Redis, deploy em Oracle Free Tier com Docker Compose. O objetivo e construir com arquitetura profissional, preparada para evoluir para SaaS no futuro.

9.2 Principios de Codigo que Devem ser Seguidos

- Clean Code: nomes descritivos, funcoes pequenas, sem comentarios obvios
- SOLID: especialmente Single Responsibility e Dependency Inversion
- Nao sobre-engenheirar: monolito modular agora, microservicos so quando necessario
- Testes: unitarios para servicos (JUnit 5 + Mockito), integracao para controllers (MockMvc)
- Migracao de banco: sempre usar Flyway — nunca `spring.jpa.hibernate.ddl-auto=create`
- DTOs sempre: nunca expor entidades JPA diretamente na API — usar record ou classe DTO
- Logs: usar SLF4J + Logback, nivel INFO em producao, DEBUG em desenvolvimento
- Tratamento de erros: `GlobalExceptionHandler` com `@ControllerAdvice`, respostas padronizadas

9.3 Convencoes de Codigo

- Java: camelCase para variaveis e metodos, PascalCase para classes, UPPER_SNAKE para constantes
- Pacotes: `com.vitapair.<feature>` (ex: `com.vitapair.nutrition.NutritionService`)
- Endpoints REST: plural para colecoes (`/foods`, `/logs`), snake_case para query params
- Respostas da API: sempre envolver em `ApiResponse<T> { data, message, success }`
- Datas: sempre usar ISO 8601 (`2025-06-21T10:30:00Z`) — nunca timestamps Unix na API
- React: componentes funcionais com hooks, nenhum componente de classe
- CSS: Tailwind utility classes, evitar CSS customizado exceto para animacoes complexas

9.4 Ordem Sugerida de Implementacao

1. Setup inicial: repositório Git, estrutura de pastas, Docker Compose com PostgreSQL + Redis

2. Backend base: Spring Boot config, Security, JWT, GlobalExceptionHandler, ApiResponse
3. Migrations Flyway: criar todas as tabelas descritas na Secao 4
4. Auth: /register e /login funcionando com JWT, testes de integracao
5. Perfil e TDEE: UserController, UserService, TdeeService com calculo Harris-Benedict
6. Sistema de Par: PairService, convite, aceitacao, criacao do tenant
7. Nutricao: Open Food Facts client, FoodLog CRUD, dashboard diario
8. Atividade: ActivityLog, calculo de calorias gastas, input de passos
9. Gamificacao: streaks, badges basicos, placar semanal
10. Frontend: setup React + TypeScript + Tailwind, React Router, axios com interceptors
11. Telas: Login/Register -> Onboarding -> Dashboard -> Log alimentar -> Feed do par
12. Deploy: Dockerfile para backend e frontend, docker-compose.yml completo, Nginx config, SSL
13. CI/CD: GitHub Actions pipeline para deploy automatico no Oracle

9.5 Variaveis de Ambiente Necessarias

Variavel	Descricao	Exemplo
DATABASE_URL	JDBC URL do PostgreSQL	jdbc:postgresql://postgres:5432/vitapair
DATABASE_USER	Usuario do banco	vitapair
DATABASE_PASSWORD	Senha do banco	gerada aleatoriamente
REDIS_HOST	Host do Redis	redis
REDIS_PASSWORD	Senha do Redis	gerada aleatoriamente
JWT_SECRET	Chave para assinar JWT (min 256 bits)	base64 aleatorio
JWT_ACCESS_EXPIRATION_MS	Expiracao do access token	900000 (15 min)
JWT_REFRESH_EXPIRATION_MS	Expiracao do refresh token	2592000000 (30 dias)

GOOGLE_CLIENT_ID	OAuth2 Google	do Google Cloud Console
GOOGLE_CLIENT_SECRET	OAuth2 Google	do Google Cloud Console
ANTHROPIC_API_KEY	Para geracao de planos por IA	sk-ant-...
FRONTEND_URL	Para CORS	https://vitapair.duckdns.org
OPEN_FOOD_FACTS_USER_AGENT	User-Agent obrigatorio	VitaPair/1.0 (seu@email.com)

10. Roadmap de Desenvolvimento

Fase 1 — MVP Funcional (para o casal)

- Auth completo (email + Google OAuth2)
- Perfil com TDEE calculado automaticamente
- Sistema de par com convite
- Registro de refeicoes via Open Food Facts
- Input manual de calorias gastas (passos + treino)
- Dashboard diario: consumido vs gasto vs meta
- Feed basico do par (ver o que o outro logou hoje)
- Deploy funcional no Oracle Free Tier com HTTPS

Fase 2 — Engajamento

- Planos de treino gerados por IA (Anthropic API)
- Plano alimentar semanal por IA
- Gamificacao completa: badges, streaks, placar semanal
- Desafios semanais entre o par
- Notificacoes push no browser
- Historico e graficos de progresso (peso, calorias, macros)
- Modo privado para refeicoes e treinos

Fase 3 — SaaS

- Multi-tenancy completo validado (testes de isolamento)
- Stripe integration: planos freemium e premium
- Landing page publica de captacao
- Sistema de onboarding guiado para novos casais
- Integracao Google Fit / Apple Health / Strava via OAuth2
- Integracao WeWard quando API disponivel
- Admin panel: metricas de uso, usuarios, receita
- Migracao de infra: Oracle -> Hetzner VPS dedicada
- Dominio proprio (.app ou .com.br)

11. pom.xml Final (pronto para usar)

Este e o pom.xml completo e validado para o projeto. Substitua o arquivo gerado pelo Spring Initializr por este. Todas as dependencias estao na versao correta e o maven-compiler-plugin esta configurado para Lombok e MapStruct coexistirem sem conflito.

11.1 Resumo das dependencias

Dependencia	Versao	Finalidade
spring-boot-starter-web	gerenciada pelo Boot	API REST com Spring MVC
spring-boot-starter-security	gerenciada pelo Boot	Autenticacao e autorizacao
spring-boot-starter-oauth2-client	gerenciada pelo Boot	Login com Google/Apple
spring-boot-starter-oauth2-resource-server	gerenciada pelo Boot	Validacao de JWT pelo Spring Security
jjwt-api / jjwt-impl / jjwt-jackson	0.12.6	Geracao e validacao de tokens JWT
spring-boot-starter-data-jpa	gerenciada pelo Boot	ORM com Hibernate
postgresql	gerenciada pelo Boot	Driver JDBC do PostgreSQL
flyway-core + flyway-database-postgresql	gerenciada pelo Boot	Versionamento de schema do banco
spring-boot-starter-data-redis	gerenciada pelo Boot	Acesso ao Redis (cache, streaks, rate limiting)
spring-session-data-redis	gerenciada pelo Boot	Sessoes armazenadas no Redis
spring-boot-starter-validation	gerenciada pelo Boot	@Valid, @NotNull, @Size nos DTOs
spring-boot-starter-actuator	gerenciada pelo Boot	Health check em /actuator/health
spring-cloud-starter-openfeign	gerenciada pelo Cloud BOM	Cliente HTTP declarativo para Open Food Facts e Anthropic API
springdoc-openapi-starter-webmvc-ui	2.6.0	Swagger UI em /swagger-ui.html
mapstruct	1.6.2	Conversao automatica entidade <-> DTO

lombok	gerenciada pelo Boot	@Getter, @Builder, @Slf4j e outros
spring-boot-devtools	gerenciada pelo Boot	Hot reload em desenvolvimento
spring-boot-docker-compose	gerenciada pelo Boot	Sobe docker-compose automaticamente no dev
spring-boot-starter-test	gerenciada pelo Boot	JUnit 5, Mockito, MockMvc
spring-security-test	gerenciada pelo Boot	Utilitarios de teste para Spring Security

11.2 Pontos criticos do build

Atencao ao maven-compiler-plugin

- Lombok DEVE vir antes do MapStruct nos annotationProcessorPaths — ordem e obrigatoria
- lombok-mapstruct-binding (0.2.0) e a "ponte" que faz os dois coexistirem — sem ela o MapStruct nao enxerga os getters do Lombok
- -Amapstruct.defaultComponentModel=spring faz os mappers virarem beans Spring (@Component automatico)
- O Lombok esta excluido do fat JAR no spring-boot-maven-plugin (correto — e so annotation processor)
- jjwt-impl e jjwt-jackson sao scope runtime — so precisam estar no classpath em execucao, nao em compilacao

11.3 Arquivo pom.xml completo

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
    https://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>

  <parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>3.5.15</version>
    <relativePath/>
  </parent>

  <groupId>com.aps</groupId>
  <artifactId>vital-pair</artifactId>
  <version>0.0.1-SNAPSHOT</version>
  <name>vital-pair</name>
  <description>VitaPair - Health and fitness app for couples</description>
```

```

<properties>
  <java.version>17</java.version>
  <spring-cloud.version>2025.0.3</spring-cloud.version>
  <jjwt.version>0.12.6</jjwt.version>
  <springdoc.version>2.6.0</springdoc.version>
  <mapstruct.version>1.6.2</mapstruct.version>
</properties>

<dependencies>

  <!-- WEB -->
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
  </dependency>

  <!-- SEGURANCA -->
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-security</artifactId>
  </dependency>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-oauth2-client</artifactId>
  </dependency>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-oauth2-resource-server</artifactId>
  </dependency>

  <!-- JWT -->
  <dependency>
    <groupId>io.jsonwebtoken</groupId>
    <artifactId>jjwt-api</artifactId>
    <version>${jjwt.version}</version>
  </dependency>
  <dependency>
    <groupId>io.jsonwebtoken</groupId>
    <artifactId>jjwt-impl</artifactId>
    <version>${jjwt.version}</version>
    <scope>runtime</scope>
  </dependency>
  <dependency>
    <groupId>io.jsonwebtoken</groupId>
    <artifactId>jjwt-jackson</artifactId>
    <version>${jjwt.version}</version>
    <scope>runtime</scope>
  </dependency>

  <!-- BANCO DE DADOS -->
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-jpa</artifactId>
  </dependency>
  <dependency>
    <groupId>org.postgresql</groupId>
    <artifactId>postgresql</artifactId>
    <scope>runtime</scope>
  </dependency>
  <dependency>
    <groupId>org.flywaydb</groupId>

```

```

    <artifactId>flyway-core</artifactId>
</dependency>
<dependency>
    <groupId>org.flywaydb</groupId>
    <artifactId>flyway-database-postgresql</artifactId>
</dependency>

<!-- REDIS -->
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-redis</artifactId>
</dependency>
<dependency>
    <groupId>org.springframework.session</groupId>
    <artifactId>spring-session-data-redis</artifactId>
</dependency>

<!-- VALIDACAO -->
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-validation</artifactId>
</dependency>

<!-- ACTUATOR -->
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-actuator</artifactId>
</dependency>

<!-- OPENFEIGN -->
<dependency>
    <groupId>org.springframework.cloud</groupId>
    <artifactId>spring-cloud-starter-openfeign</artifactId>
</dependency>

<!-- SPRINGDOC / SWAGGER UI -->
<dependency>
    <groupId>org.springdoc</groupId>
    <artifactId>springdoc-openapi-starter-webmvc-ui</artifactId>
    <version>${springdoc.version}</version>
</dependency>

<!-- MAPSTRUCT -->
<dependency>
    <groupId>org.mapstruct</groupId>
    <artifactId>mapstruct</artifactId>
    <version>${mapstruct.version}</version>
</dependency>

<!-- LOMBOK -->
<dependency>
    <groupId>org.projectlombok</groupId>
    <artifactId>lombok</artifactId>
    <optional>true</optional>
</dependency>

<!-- DEV -->
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-devtools</artifactId>
    <scope>runtime</scope>

```

```

        <optional>true</optional>
    </dependency>
</dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-docker-compose</artifactId>
    <scope>runtime</scope>
    <optional>true</optional>
</dependency>

<!-- TESTES -->
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-test</artifactId>
    <scope>test</scope>
</dependency>
<dependency>
    <groupId>org.springframework.security</groupId>
    <artifactId>spring-security-test</artifactId>
    <scope>test</scope>
</dependency>

</dependencies>

<dependencyManagement>
    <dependencies>
        <dependency>
            <groupId>org.springframework.cloud</groupId>
            <artifactId>spring-cloud-dependencies</artifactId>
            <version>${spring-cloud.version}</version>
            <type>pom</type>
            <scope>import</scope>
        </dependency>
    </dependencies>
</dependencyManagement>

<build>
    <plugins>
        <plugin>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-maven-plugin</artifactId>
            <configuration>
                <excludes>
                    <exclude>
                        <groupId>org.projectlombok</groupId>
                        <artifactId>lombok</artifactId>
                    </exclude>
                </excludes>
            </configuration>
        </plugin>
        <plugin>
            <groupId>org.apache.maven.plugins</groupId>
            <artifactId>maven-compiler-plugin</artifactId>
            <configuration>
                <annotationProcessorPaths>
                    <!-- 1. Lombok primeiro (obrigatorio) -->
                    <path>
                        <groupId>org.projectlombok</groupId>
                        <artifactId>lombok</artifactId>
                    </path>
                    <!-- 2. MapStruct depois -->
                    <path>
                        <groupId>org.mapstruct</groupId>

```

```
        <artifactId>mapstruct-processor</artifactId>
        <version>${mapstruct.version}</version>
    </path>
    <!-- 3. Ponte Lombok+MapStruct -->
    <path>
        <groupId>org.projectlombok</groupId>
        <artifactId>lombok-mapstruct-binding</artifactId>
        <version>0.2.0</version>
    </path>
</annotationProcessorPaths>
<compilerArgs>
    <arg>-Amapstruct.defaultComponentModel=spring</arg>
</compilerArgs>
</configuration>
</plugin>
</plugins>
</build>

</project>
```